LAB GUIDELINES

Lab 12 — Performance Monitoring, Baselining, Event Logs

Maps to CompTIA Server+ objective **2.3** — Monitoring (uptime, thresholds, performance — memory, disk IOPS, capacity vs. utilisation, network, CPU; event logs — configuration, shipping; alerting, reporting, retention, rotation; baselining).

Run all commands on https://killercoda.com/playgrounds/scenario/ubuntu

Required software (free, apt): `sysstat`, `htop`, `iotop`, `fio`, `prometheus-node-exporter`

```
apt update && apt install -y sysstat htop iotop fio prometheus-node-exporter
systemctl enable --now sysstat
sed -i 's/^ENABLED=.*/ENABLED="true"/' /etc/default/sysstat 2>/dev/null
```

Step 1 — Uptime, the simplest health metric

```
uptime
who -b
last reboot | head
```

`uptime` reports load averages (1, 5, 15 min). On an N-CPU box, sustained load > N means the system is overloaded.

Step 2 — CPU baseline

```
mpstat 1 5
sar -u 1 5
```

Capture **idle %, user %, system %, iowait %**. iowait climbing is a storage problem, not a CPU problem.

Step 3 — Memory baseline

```
free -h
vmstat 1 5
sar -r 1 5
```

Track **available** (not "free"), **swap usage**, **page-in/page-out**.

Step 4 — Disk IOPS, throughput, utilisation

```
iostat -xz 1 5
# Generate load:
fio --name=ioptest --filename=/tmp/iotest --size=128M --bs=4k --rw=randwrite \
    --ioengine=libaio --iodepth=16 --numjobs=2 --time_based=1 --runtime=10 --group_reporting
iostat -xz 1 3
```

Server+ specifically calls out **IOPS** and **capacity vs. utilisation**. From `iostat -x`: `%util` is utilisation, `r/s + w/s` is IOPS, `await` is latency.

Step 5 — Network baseline

```
sar -n DEV 1 5
ss -s
```

Step 6 — Establish a baseline file

A baseline is what "normal" looks like, so you can detect deviation later.

```
mkdir -p /var/log/baseline
{
  echo "# Baseline captured $(date -Is) on $(hostname -f)"
  echo "## CPU"; mpstat 1 3
```

```
  echo "## Memory"; free -h
  echo "## Disk"; iostat -xz 1 3
  echo "## Net"; sar -n DEV 1 3
} > /var/log/baseline/baseline-$(date +%F).txt
ls /var/log/baseline/
```

Re-capture monthly. Document where files live in your runbook (Lab 17).

Step 7 — Event log inspection

```
journalctl --since "1 hour ago" -p err
journalctl -u nginx -n 50
tail -n 50 /var/log/syslog
```

Server+ 2.3 calls out **Event logs: configuration, shipping**. Configure rotation:

```
cat /etc/logrotate.conf | head
ls /etc/logrotate.d/
```

Default policy rotates weekly, keeps 4 weeks. Tune **retention** to match the audit / compliance policy (Lab 22).

Step 8 — Log shipping (centralised collection)

Edit `/etc/rsyslog.conf` to forward to a SIEM (example syntax — don't enable without a receiver):

```
echo '*.* @@siem.lab.local:514' > /etc/rsyslog.d/90-forward.conf
# systemctl restart rsyslog
```

@@ = TCP (reliable). A single @ = UDP (lossy, only for very high volume).

Step 9 — Thresholds + alerting (using Prometheus node_exporter)

```
 apt install -y prometheus-node-exporter
systemctl enable --now prometheus-node-exporter
curl -s http://127.0.0.1:9100/metrics | grep -E 'node_cpu_seconds_total|node_memory_MemAvailable|no
```

Wire this to a Prometheus scrape + Alertmanager rule. Example alert (PromQL):

```
ALERT HighCPU
  IF avg by (instance)(rate(node_cpu_seconds_total{mode!="idle"}[5m])) > 0.85
  FOR 10m
  ANNOTATIONS { summary = "CPU > 85% for 10 minutes" }
```

Step 10 — Reporting

Aggregate the day's `sar` data:

```
sar -u   | head -5
sar -r   | head -5
sar -dp  | head
```

Trend these into a weekly report — the Server+ exam expects monitoring → reporting → thresholds → alerting as a connected loop.

Free online tools

- **sysstat docs** — http://sebastien.godard.pagesperso-orange.fr/
- **Prometheus node_exporter** — https://github.com/prometheus/node_exporter
- **Grafana dashboard library** — https://grafana.com/grafana/dashboards/

What you learned

- The four classes of metrics Server+ tests: CPU, memory, disk IOPS, network.
- Capturing a baseline file you can diff against later.

- Event log inspection, rotation, retention, and shipping to a central collector.
- Threshold → alert wiring with Prometheus node_exporter.